

Railway Transport & Reservation Management System

Database Project — Phase 2 Submission

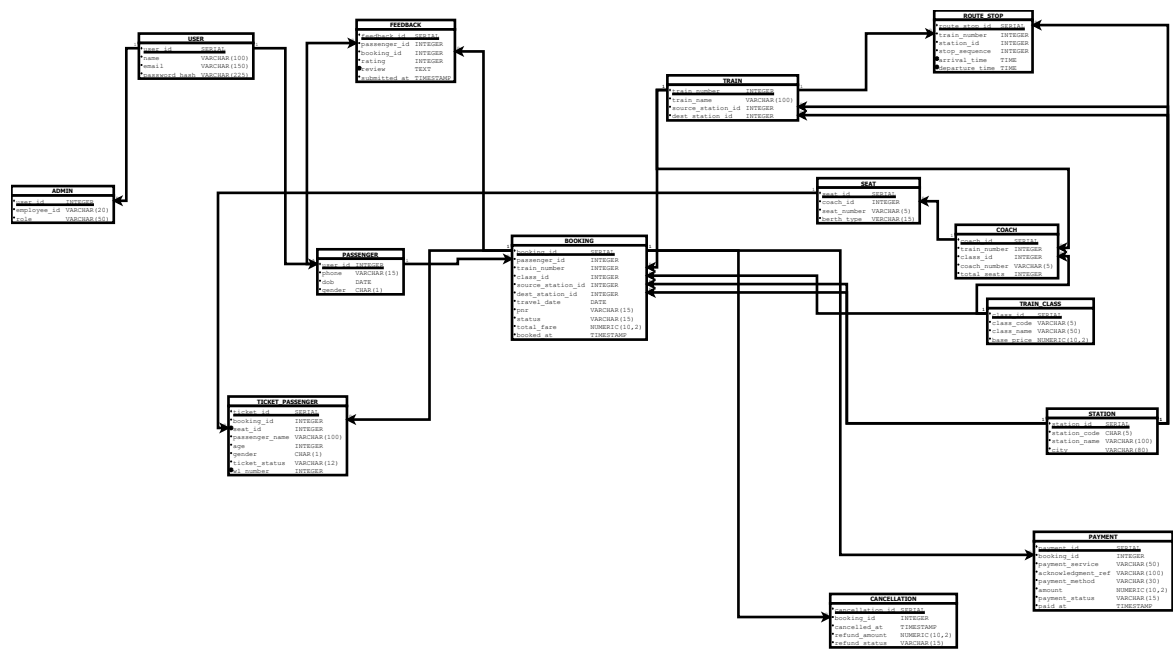
student ids of members

202401024
202401068
202401049
202401059

Submission Contents:

- (1) Normalized Relational Schema Diagram
- (2) Minimal Functional Dependency Set
- (3) Normalization Proofs (BCNF)
- (4) DDL Scripts for PostgreSQL

PART:1 Relational Schema Diagram



Part 2: Minimal Functional Dependency Set

A minimal (canonical) FD set has: (a) single attribute on right-hand side, (b) no extraneous attributes on left-hand side, and (c) no redundant FDs. Below is the complete minimal FD set for all 14 relations.

USER

```
user_id → name
user_id → email
user_id → password_hash
email → user_id    [alternate key]
```

PASSENGER

```
user_id → phone
user_id → dob
user_id → gender
```

ADMIN

```
user_id → employee_id
user_id → role
employee_id → user_id    [alternate key]
```

STATION

```
station_id → station_code
station_id → station_name
station_id → city
station_code → station_id    [alternate key]
```

TRAIN

```
train_number → train_name
train_number → source_station_id
train_number → dest_station_id
```

TRAIN_CLASS

```
class_id → class_code
class_id → class_name
class_id → base_price
```

class_code → class_id [alternate key]

ROUTE_STOP

{train_number, stop_sequence} → station_id
{train_number, stop_sequence} → arrival_time
{train_number, stop_sequence} → departure_time
{train_number, station_id} → stop_sequence [alt key]

COACH

coach_id → train_number
coach_id → class_id
coach_id → coach_number
coach_id → total_seats
{train_number, coach_number} → coach_id [alt key]

SEAT

seat_id → coach_id
seat_id → seat_number
seat_id → berth_type
{coach_id, seat_number} → seat_id [alt key]

BOOKING

booking_id → passenger_id
booking_id → train_number
booking_id → class_id
booking_id → source_station_id
booking_id → dest_station_id
booking_id → travel_date
booking_id → pnr
booking_id → status
booking_id → total_fare
booking_id → booked_at
pnr → booking_id [alternate key]

TICKET_PASSENGER

ticket_id → booking_id
ticket_id → seat_id
ticket_id → passenger_name
ticket_id → age

ticket_id → gender
ticket_id → ticket_status
ticket_id → wl_number

PAYMENT

payment_id → booking_id
payment_id → payment_service
payment_id → acknowledgment_ref
payment_id → payment_method
payment_id → amount
payment_id → payment_status
payment_id → paid_at
booking_id → payment_id [alternate key, 1-to-1]

CANCELLATION

cancellation_id → booking_id
cancellation_id → cancelled_at
cancellation_id → refund_amount
cancellation_id → refund_status
booking_id → cancellation_id [alternate key]

FEEDBACK

feedback_id → passenger_id
feedback_id → booking_id
feedback_id → rating
feedback_id → review
feedback_id → submitted_at
booking_id → feedback_id [alternate key, 0..1]

Part 3: Normalization Proofs (BCNF)

As per the professor's definition: A relation R is in BCNF when, for every FD $X \rightarrow Y$ that holds on R, X is a superkey of R. We apply this check to every relation in our schema.

3.1 USER

Relation	USER(user_id, name, email, password_hash)
Key(s)	user_id [also email is alternate key]
Functional Dependencies	user_id → name, user_id → email, user_id → password_hash, email → user_id
In BCNF?	YES — In BCNF
Reason	Both user_id and email are keys. Every FD has a key as its determinant. No FD violates BCNF.

3.2 PASSENGER

Relation	PASSENGER(user_id, phone, dob, gender)
Key(s)	user_id
Functional Dependencies	user_id → phone, user_id → dob, user_id → gender
In BCNF?	YES — In BCNF
Reason	user_id is the only key. Every FD has user_id as determinant. BCNF satisfied.

3.3 ADMIN

Relation	ADMIN(user_id, employee_id, role)
Key(s)	user_id [also employee_id is alternate key]
Functional Dependencies	user_id → employee_id, user_id → role, employee_id → user_id
In BCNF?	YES — In BCNF
Reason	Both user_id and employee_id are keys. All FD determinants are keys. BCNF satisfied.

3.4 STATION

Relation	STATION(station_id, station_code, station_name, city)
-----------------	---

Key(s)	station_id [also station_code is alternate key]
Functional Dependencies	station_id→station_code, station_id→station_name, station_id→city, station_code→station_id
In BCNF?	YES — In BCNF
Reason	Both station_id and station_code are keys. Every determinant is a key. BCNF satisfied.

3.5 TRAIN

Relation	TRAIN(train_number, train_name, source_station_id, dest_station_id)
Key(s)	train_number
Functional Dependencies	train_number→train_name, train_number→source_station_id, train_number→dest_station_id
In BCNF?	YES — In BCNF
Reason	train_number is the sole key. All FDs have train_number as determinant. BCNF satisfied.

3.6 TRAIN_CLASS

Relation	TRAIN_CLASS(class_id, class_code, class_name, base_price)
Key(s)	class_id [also class_code is alternate key]
Functional Dependencies	class_id→class_code, class_id→class_name, class_id→base_price, class_code→class_id
In BCNF?	YES — In BCNF
Reason	Both class_id and class_code are keys. Every FD determinant is a key. BCNF satisfied.

3.7 ROUTE_STOP

Relation	ROUTE_STOP(route_stop_id, train_number, station_id, stop_sequence, arrival_time, departure_time)
Key(s)	{train_number, stop_sequence} [also {train_number, station_id}; route_stop_id is surrogate key]
Functional Dependencies	{train_number, stop_sequence}→station_id, →arrival_time, →departure_time; {train_number, station_id}→stop_sequence
In BCNF?	YES — In BCNF
Reason	All three candidate keys (surrogate route_stop_id, composite {train_number, stop_sequence}, composite {train_number, station_id}) are

	determinants of their respective FDs. No non-key FD exists. BCNF satisfied.
--	---

3.8 COACH

Relation	COACH(coach_id, train_number, class_id, coach_number, total_seats)
Key(s)	coach_id [also {train_number, coach_number} is alternate key]
Functional Dependencies	coach_id→train_number, coach_id→class_id, coach_id→coach_number, coach_id→total_seats, {train_number,coach_number}→coach_id
In BCNF?	YES — In BCNF
Reason	coach_id and {train_number, coach_number} are both keys. Every FD has a key as determinant. BCNF satisfied.

3.9 SEAT

Relation	SEAT(seat_id, coach_id, seat_number, berth_type)
Key(s)	seat_id [also {coach_id, seat_number} is alternate key]
Functional Dependencies	seat_id→coach_id, seat_id→seat_number, seat_id→berth_type, {coach_id,seat_number}→seat_id
In BCNF?	YES — In BCNF
Reason	Both seat_id and {coach_id, seat_number} are keys. All FD determinants are keys. BCNF satisfied.

3.10 BOOKING

Relation	BOOKING(booking_id, passenger_id, train_number, class_id, source_station_id, dest_station_id, travel_date, pnr, status, total_fare, booked_at)
Key(s)	booking_id [also pnr is alternate key]
Functional Dependencies	booking_id→{all other attributes}, pnr→booking_id
In BCNF?	YES — In BCNF
Reason	booking_id and pnr are both keys. Every non-trivial FD has a key as determinant. No partial or transitive dependency exists. BCNF satisfied.

3.11 TICKET_PASSENGER

Relation	TICKET_PASSENGER(ticket_id, booking_id, seat_id, passenger_name, age, gender, ticket_status, wl_number)
-----------------	---

Key(s)	ticket_id
Functional Dependencies	ticket_id → {all other attributes}
In BCNF?	YES — In BCNF
Reason	ticket_id is the only key. Every attribute depends only on ticket_id. No non-trivial FD violates BCNF. BCNF satisfied.

3.12 PAYMENT

Relation	PAYMENT(payment_id, booking_id, payment_service, acknowledgment_ref, payment_method, amount, payment_status, paid_at)
Key(s)	payment_id [also booking_id is alternate key — 1-to-1 with BOOKING]
Functional Dependencies	payment_id → {all}, booking_id → payment_id
In BCNF?	YES — In BCNF
Reason	Both payment_id and booking_id are keys. All FD determinants are keys. BCNF satisfied.

3.13 CANCELLATION

Relation	CANCELLATION(cancellation_id, booking_id, cancelled_at, refund_amount, refund_status)
Key(s)	cancellation_id [also booking_id is alternate key]
Functional Dependencies	cancellation_id → {all}, booking_id → cancellation_id
In BCNF?	YES — In BCNF
Reason	Both cancellation_id and booking_id are keys. Every FD determinant is a key. BCNF satisfied.

3.14 FEEDBACK

Relation	FEEDBACK(feedback_id, passenger_id, booking_id, rating, review, submitted_at)
Key(s)	feedback_id [booking_id is alternate key — at most one feedback per booking]
Functional Dependencies	feedback_id → {all}, booking_id → feedback_id
In BCNF?	YES — In BCNF

Reason

Both feedback_id and booking_id are keys. All FD determinants are keys.
BCNF satisfied.

Conclusion: All 14 relations are in BCNF. No decomposition is required. The schema was designed bottom-up from the ER diagram, ensuring each entity and relationship was kept in a single clean relation with no transitive or partial dependencies.

Part 4: DDL Scripts — PostgreSQL

The following script creates a dedicated schema named **tts** (Train Transport System) on the PostgreSQL server. Run this script as a single file.

Schema Creation

```
-- =====
-- Railway Transport & Reservation System
-- DDL Script - PostgreSQL
-- Schema: tts
-- Group: 202401068
-- =====

DROP SCHEMA IF EXISTS tts CASCADE;
CREATE SCHEMA tts;
SET search_path TO tts;
```

Table: USER

```
CREATE TABLE tts."user" (
    user_id      SERIAL          PRIMARY KEY,
    name         VARCHAR(100)    NOT NULL,
    email        VARCHAR(150)    NOT NULL UNIQUE,
    password_hash VARCHAR(255)    NOT NULL
);
```

Table: PASSENGER

```
CREATE TABLE tts.passenger (
    user_id  INTEGER          PRIMARY KEY
            REFERENCES tts."user"(user_id)
            ON DELETE CASCADE,

    phone    VARCHAR(15) NOT NULL,
    dob      DATE          NOT NULL,
    gender    CHAR(1)      CHECK (gender IN ('M','F','O'))
);
```

Table: ADMIN

```
CREATE TABLE tts.admin (
    user_id  INTEGER          PRIMARY KEY
            REFERENCES tts."user"(user_id)
            ON DELETE CASCADE,
```

```
    employee_id VARCHAR(20) NOT NULL UNIQUE,  
    role        VARCHAR(50) NOT NULL  
);
```

Table: STATION

```
CREATE TABLE tts.station (  
    station_id  SERIAL          PRIMARY KEY,  
    station_code CHAR(5)        NOT NULL UNIQUE,  
    station_name VARCHAR(100) NOT NULL,  
    city        VARCHAR(80) NOT NULL  
);
```

Table: TRAIN

```
CREATE TABLE tts.train (  
    train_number    INTEGER          PRIMARY KEY,  
    train_name      VARCHAR(100) NOT NULL,  
    source_station_id INTEGER        NOT NULL  
                                REFERENCES tts.station(station_id),  
    dest_station_id INTEGER          NOT NULL  
                                REFERENCES tts.station(station_id)  
);
```

Table: TRAIN_CLASS

```
CREATE TABLE tts.train_class (  
    class_id  SERIAL          PRIMARY KEY,  
    class_code VARCHAR(5)      NOT NULL UNIQUE,  
    class_name VARCHAR(50)     NOT NULL,  
    base_price NUMERIC(10,2) NOT NULL  
);
```

Table: ROUTE_STOP

```
CREATE TABLE tts.route_stop (  
    route_stop_id SERIAL      PRIMARY KEY,  
    train_number    INTEGER NOT NULL  
                                REFERENCES tts.train(train_number),  
    station_id      INTEGER NOT NULL  
                                REFERENCES tts.station(station_id),  
    stop_sequence   INTEGER NOT NULL,  
    arrival_time    TIME,  
    departure_time  TIME,  
    UNIQUE (train_number, stop_sequence),
```

```

        UNIQUE (train_number, station_id)
    );

```

Table: COACH

```

CREATE TABLE tts.coach (
    coach_id      SERIAL          PRIMARY KEY,
    train_number  INTEGER          NOT NULL
                        REFERENCES tts.train(train_number),
    class_id      INTEGER          NOT NULL
                        REFERENCES tts.train_class(class_id),
    coach_number  VARCHAR(5)       NOT NULL,
    total_seats   INTEGER          NOT NULL,
    UNIQUE (train_number, coach_number)
);

```

Table: SEAT

```

CREATE TABLE tts.seat (
    seat_id       SERIAL          PRIMARY KEY,
    coach_id      INTEGER          NOT NULL
                        REFERENCES tts.coach(coach_id),
    seat_number   VARCHAR(5)       NOT NULL,
    berth_type    VARCHAR(15)      NOT NULL,
    UNIQUE (coach_id, seat_number)
);

```

Table: BOOKING

```

CREATE TABLE tts.booking (
    booking_id    SERIAL          PRIMARY KEY,
    passenger_id  INTEGER          NOT NULL
                        REFERENCES tts.passenger(user_id),
    train_number  INTEGER          NOT NULL
                        REFERENCES tts.train(train_number),
    class_id      INTEGER          NOT NULL
                        REFERENCES tts.train_class(class_id),
    source_station_id INTEGER      NOT NULL
                        REFERENCES tts.station(station_id),
    dest_station_id INTEGER        NOT NULL
                        REFERENCES tts.station(station_id),
    travel_date   DATE             NOT NULL,
    pnr           VARCHAR(15)       NOT NULL UNIQUE,
    status        VARCHAR(15)       NOT NULL
                        CHECK (status IN ('CONFIRMED', 'WAITING', 'CANCELLED')),
    total_fare     NUMERIC(10,2)    NOT NULL,

```

```

        booked_at          TIMESTAMP          DEFAULT NOW()
    );

```

Table: TICKET_PASSENGER (weak entity)

```

CREATE TABLE tts.ticket_passenger (
    ticket_id          SERIAL          PRIMARY KEY,
    booking_id         INTEGER          NOT NULL
                        REFERENCES tts.booking(booking_id),
    seat_id            INTEGER
                        REFERENCES tts.seat(seat_id),
    passenger_name     VARCHAR(100) NOT NULL,
    age                INTEGER          NOT NULL,
    gender              CHAR(1)          NOT NULL
                        CHECK (gender IN ('M','F','O')),
    ticket_status       VARCHAR(12) NOT NULL
                        CHECK (ticket_status IN ('CONFIRMED','WL','CANCELLED')),
    wl_number           INTEGER
);

```

Table: PAYMENT

```

CREATE TABLE tts.payment (
    payment_id          SERIAL          PRIMARY KEY,
    booking_id         INTEGER          NOT NULL UNIQUE
                        REFERENCES tts.booking(booking_id),
    payment_service     VARCHAR(50)     NOT NULL,
    acknowledgment_ref  VARCHAR(100) NOT NULL,
    payment_method       VARCHAR(30)     NOT NULL,
    amount              NUMERIC(10,2) NOT NULL,
    payment_status       VARCHAR(15)     NOT NULL
                        CHECK (payment_status IN
('SUCCESS','PENDING','FAILED')),
    paid_at             TIMESTAMP          DEFAULT NOW()
);

```

Table: CANCELLATION

```

CREATE TABLE tts.cancellation (
    cancellation_id     SERIAL          PRIMARY KEY,
    booking_id          INTEGER          NOT NULL UNIQUE
                        REFERENCES tts.booking(booking_id),
    cancelled_at         TIMESTAMP          DEFAULT NOW(),
    refund_amount        NUMERIC(10,2) NOT NULL,
    refund_status        VARCHAR(15)     NOT NULL
                        CHECK (refund_status IN ('PENDING','PROCESSED','FAILED'))
);

```

```
);
```

Table: FEEDBACK

```
CREATE TABLE tts.feedback (  
    feedback_id SERIAL      PRIMARY KEY,  
    passenger_id INTEGER    NOT NULL  
        REFERENCES tts.passenger(user_id),  
    booking_id  INTEGER    NOT NULL UNIQUE  
        REFERENCES tts.booking(booking_id),  
    rating      INTEGER    CHECK (rating BETWEEN 1 AND 5),  
    review      TEXT,  
    submitted_at TIMESTAMP DEFAULT NOW()  
);
```

Indexes for Performance

```
-- Speed up train search by route  
CREATE INDEX idx_route_stop_train  
    ON tts.route_stop(train_number, stop_sequence);  
  
-- Speed up PNR lookups  
CREATE INDEX idx_booking_pnr  
    ON tts.booking(pnr);  
  
-- Speed up passenger booking history  
CREATE INDEX idx_booking_passenger  
    ON tts.booking(passenger_id);  
  
-- Speed up waiting list queries per train/class/date  
CREATE INDEX idx_ticket_status  
    ON tts.ticket_passenger(booking_id, ticket_status);  
  
-- Speed up seat availability check  
CREATE INDEX idx_seat_coach  
    ON tts.seat(coach_id);
```

End of DDL Script

```
-- All tables created in schema tts  
-- Total tables: 14  
-- All relations proven to be in BCNF  
-- =====
```